

Learn Low-Level Design in 25 Days: Day 16— Database Layer Design: Repository & DAO Patterns

(From the “Learn Low-Level Design in 25 Days” Series— save the full list here)

TechPreneur

Published October 28, 2025

Free: No

Read on Freedium · original

Contents

The Everyday Developer Struggle: "Where Should My DB Logic Go?" 0

Understanding the Problem: Mixing Business and Data Logic 0

Step 1: Meet the DAO Pattern — The Data Gatekeeper 0

Benefits 0

Step 2: Repository Pattern — Going One Level Higher 0

A Quick Visual 0

Real-World Case Study: How Netflix Does It 0

Try It Yourself 0

Key Takeaways 0

What's Next? 0

The Everyday Developer Struggle: "Where Should My DB Logic Go?"

You've probably been there. You're building an app — say a Bookstore API — and suddenly you realize your service classes are full of SQL queries:

```
public class BookService {
    public Book getBookById(int id) {
        Connection conn = Database.getConnection();
        PreparedStatement ps = conn.prepareStatement("SELECT * FROM books WHERE id=?");
        // ...
    }
}
```

It starts simple. Then, as your app grows, you find yourself repeating queries, copy-pasting connection logic, and struggling to change your database schema without breaking half your codebase.

That's when most developers realize they need **a proper database layer** — one that isolates persistence logic from business logic.



Welcome to **Day 16 of "Learn Low-Level Design in 25 Days."** Today, we'll demystify the **Repository** and **DAO (Data Access Object)** patterns — two unsung heroes that make your codebase clean, scalable, and testable.

By the end of this article, you'll know:

- Why your service classes should never talk directly to the database
- How to design a flexible, layered data access structure
- The subtle but important difference between DAO and Repository patterns
- And how large systems like Netflix or Amazon structure their persistence layer

Understanding the Problem: Mixing Business and Data Logic

Let's take a simple example.

Suppose you're building a small library system. You start with a `BookService` that handles both business and database logic:

```
public class BookService {
    public void addBook(Book book) {
        Connection conn = Database.getConnection();
        PreparedStatement ps = conn.prepareStatement("INSERT INTO books VALUES (?, ?, ?)")
;
        ps.setInt(1, book.getId());
        ps.setString(2, book.getTitle());
        ps.setString(3, book.getAuthor());
        ps.executeUpdate();
    }

    public Book getBookById(int id) {
        Connection conn = Database.getConnection();
        PreparedStatement ps = conn.prepareStatement("SELECT * FROM books WHERE id=?");
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();
        return new Book(rs.getInt("id"), rs.getString("title"), rs.getString("author"));
    }
}
```

At first, it works fine. But as your app grows:

- Every class repeats database boilerplate
- You can't easily swap your SQL DB for MongoDB
- Unit testing becomes painful (you need an actual DB)
- Business logic and persistence logic are tangled

That's where **DAO** and **Repository** patterns come in.

Step 1: Meet the DAO Pattern — The Data Gatekeeper

The **Data Access Object (DAO)** pattern's job is simple: Encapsulate all database access in one place.

Instead of having SQL sprinkled across your codebase, all CRUD (Create, Read, Update, Delete) operations live inside a DAO class.

```
public interface BookDAO {
    void save(Book book);
    Book findById(int id);
    List<Book> findAll();
}
```

A concrete implementation might look like this:

```
public class BookDAOImpl implements BookDAO {
    private final DataSource dataSource;

    public BookDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public void save(Book book) {
        try (Connection conn = dataSource.getConnection()) {
            PreparedStatement ps = conn.prepareStatement("INSERT INTO books VALUES
(?, ?, ?)");
            ps.setInt(1, book.getId());
            ps.setString(2, book.getTitle());
            ps.setString(3, book.getAuthor());
            ps.executeUpdate();
        }
    }

    public Book findById(int id) {
        try (Connection conn = dataSource.getConnection()) {
            PreparedStatement ps = conn.prepareStatement("SELECT * FROM books WHERE id=?")
;
            ps.setInt(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return new Book(rs.getInt("id"), rs.getString("title"), rs.getString("auth
or"));
            }
        }
        return null;
    }
}
```

Now your `BookService` looks much cleaner:

```
public class BookService {
    private final BookDAO bookDAO;

    public BookService(BookDAO bookDAO) {
        this.bookDAO = bookDAO;
    }

    public void addBook(Book book) {
        bookDAO.save(book);
    }

    public Book getBookById(int id) {
        return bookDAO.findById(id);
    }
}
```

Benefits

- Database logic is isolated
- You can swap the database (SQL → NoSQL) easily
- Testing becomes simpler (you can mock the DAO)
- The service layer focuses only on **business rules**

Step 2: Repository Pattern — Going One Level Higher

The **Repository** pattern is like a smarter DAO. While a DAO deals directly with tables and SQL, a Repository manages **domain entities** and **aggregates**.

Think of it like this:

- **DAO** = Low-level CRUD on a single table
- **Repository** = Higher-level abstraction that treats multiple DAOs as one logical unit

In large systems, you might have multiple DAOs working under one repository.

```
public interface BookRepository {
    void add(Book book);
    Book getById(int id);
    List<Book> getAll();
}
```

And the implementation might use one or more DAOs internally:

```
public class BookRepositoryImpl implements BookRepository {
    private final BookDAO bookDAO;

    public BookRepositoryImpl(BookDAO bookDAO) {
        this.bookDAO = bookDAO;
    }

    public void add(Book book) {
        // Add domain validation or aggregation logic here
        bookDAO.save(book);
    }

    public Book getById(int id) {
        return bookDAO.findById(id);
    }

    public List<Book> getAll() {
        return bookDAO.findAll();
    }
}
```

This structure cleanly separates **domain logic** (Repository) from **persistence logic** (DAO).

A Quick Visual

```
Service Layer
     |
     v
Repository Layer (Business-Aware)
     |
     v
DAO Layer (Database-Aware)
     |
     v
Database
```

Or in a class hierarchy:

```
BookService
   └── BookRepository
          └── BookDAO
                 └── Database
```

Real-World Case Study: How Netflix Does It

In Netflix's microservices architecture, each service (like "User", "Playback", or "Recommendations") maintains its own **Repository layer** that abstracts away the persistence mechanism.

When Netflix migrated parts of its data from MySQL to Cassandra, the change was isolated inside the DAO layer — service and repository code remained untouched.

That's the power of clean layering.

Try It Yourself

1. **Challenge 1:** Design a `UserRepository` and `UserDAO` for an e-commerce system. Users can have multiple addresses — how would you structure your layers?
2. **Challenge 2:** Suppose you need to migrate from MySQL to MongoDB. Which layer(s) should change, and which should remain untouched?

Key Takeaways

- **DAO Pattern:** Encapsulates CRUD and persistence logic.
- **Repository Pattern:** Wraps DAOs and adds domain logic.
- Always separate **business logic** from **data access logic**.
- Makes testing, swapping databases, and scaling much easier.
- This is the foundation of clean, layered architecture in real-world systems.

What's Next?

Tomorrow in **Day 17**, we'll explore **Designing a Cache System (LRU, LFU, Write-Through)** — and see how it complements your database layer for high performance and low latency.

Follow the complete series here and save it for reference: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)